Hong Kong Metropolitan University

COMP2090SEF

Data Structures, Algorithms And Problem Solving

Course Project Report

Group 105

Submission date:

We declare that:
(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;
(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration; (iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
CHENG Siu Fung	14074189
CHAN Chun Sing	14259934
LI Zhengxuan	14037890

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

3. All members are liable to the plagiarism incidence, even if a free-rider.

Github link:
https://github.com/owounnamed/COMP-2090SEF-GRP_105-project

Abstract

UniEats, a Python-based application that allows finding and rating restaurants, is introduced. Users can search for restaurants located in the neighborhoods of Ho Man Tin and Mong Kok, filter the list by area, cuisine type, and maximum budget, sort results, submit restaurant reviews with a rating and photo, and get recommendations. Administrators can manage restaurants, create, edit, delete records, and remove any inappropriate reviews. Data is stored using SQLite databases. Besides, the system includes optional map visualization feature and images preview. Thus, this application solves the practical problem of organizing information about restaurants in one system.

1. Introduction and Problem-Solving Approach

Selecting a restaurant in real life might be challenging since information is often distributed between posts in social media, chat groups, or even conversations with friends. Moreover, information can lack completeness and be unrelated to personal needs and requirements. To solve this problem, UniEats provides a review system related specifically to two Kowloon districts relevant for the target audience – students and workers. This approach uses the following steps. First, the application gathers restaurant information into one organized database instead of unstructured posts. Then, searching options by area, cuisine type, and maximum budget are provided to users. Next, sorting of restaurant information by various factors such as rating or alphabetical order is supported. In addition, users are able to post their reviews and rate restaurants. Furthermore, recommendations feature provides a possibility to receive random suggestions or similar restaurants based on the graph theory. Finally, an administrative part is available to ensure proper information management. Overall, this solution helps organize restaurant information in one application.

2. Core Functions and Techniques

To implement the UniEats system, a set of Python modules is used including main.py, gui.py, models.py, database.py, services.py, and utils.py. The process starts from the initialization of the UniEatsApp object in the main module and running of the Tkinter event loop there. Gui module, or gui.py, is the biggest one and includes the user panel, map panel, and admin panel as a desktop application interface. With the help of the interface, users may filter the list, sort results, post their reviews with photos, preview stored information, and get restaurant recommendations. Admin panel includes logging in, adding new restaurants, updating, and deleting existing restaurant information, seeing all reviews, and deleting selected reviews. Database module, database.py, deals with database persistence, creates restaurants, reviews, and deleted reviews tables, seeds the restaurants, and supports CRUD operations. Services module, or services.py, works as a logical layer between the GUI and the database by implementing restaurant filtration, sorting, random and graph recommendations,

calculating the distance between locations, storing the review images, and displaying the restaurant marker to the map. Utils module, utils.py, contains shared resources, including constants, handling images folder, optional modules support, distance calculator using the Haversine formula, and shell sort implementation. Finally, models module, models.py, defines the key classes utilized in this application such as Person, User, Admin, Restaurant, FeaturedRestaurant, Review, and RecommendationGraph. Overall, these modules show modular programming since each of them implements one responsibility.

3. Usage of OOP Concepts

OOP concepts covered in the course are demonstrated in the following way. Firstly, classes and objects play a major role throughout the system such as Restaurant objects, Review objects, and UniEatsApp object. Secondly, abstraction is illustrated by Person class which is an abstract base class inheriting ABC and providing the abstract method – get_role(). Thirdly, inheritance is shown as User(Person), Admin(Person), and FeaturedRestaurant(Restaurant) where classes inherit from their parent classes and customize their functionality. Fourthly, polymorphism is used to implement the different behavior of subclasses, including different values returned in User.get_role() and Admin.get_role(), and overriding of the Restaurant.display_text() method in FeaturedRestaurant.display_text(). Fifthly, encapsulation in Python is provided by defining a protected attribute _username and getter and setter methods for accessing it in Person class. Such a technique ensures data safety and maintainability better than accessing attributes directly. Finally, modular programming is used by separating logic into different files, so that the GUI does not encapsulate the database operations and vice versa.

4. External Resources and Libraries

Several external and built-in Python modules were used in the project. Specifically, Python is used as the programming language. Tkinter library helps with developing a graphical interface of the application. sqlite3 library was used for persistent storage of database tables. The abc module is helpful for implementing abstractions. Optional Pillow module supports image preview. Optional tkintermapview library enables displaying a map. There are no other large frameworks needed for implementing this solution. All the mentioned libraries meet the needs of the project – developing GUI interface, storing data, implementing abstractions, optional image support, and optional map visualization.

5. Demonstration Examples and Test Cases

There are several test scenarios used to demonstrate the application functionality. In the first scenario, area = Mong Kok, cuisine = Japanese, and budget = 200 are selected to show targeted search of restaurant. In the second scenario, results are sorted by rating to sort restaurants in the descending order based on their rating value. In the third scenario, a review with username, rating, comment, and optional photo is submitted. As a result, the review is stored in the SQLite database and shown on the GUI page with reviews. The fourth scenario shows that the wrong input value such as rating = 7 raises an error and stops reviewing procedure. In the fifth scenario, random recommendation is requested by clicking the corresponding button, and only one restaurant is recommended among the current list. Finally, the sixth scenario illustrates the administrative process by showing that a deleted review was copied to the deleted reviews history table.

6. Weaknesses and Future Improvements

Although this project meets the requirements specified at the beginning, there are several disadvantages revealed during testing. The first weakness is related to the admin login where only username and password are used, thus, the security level is very low. Graph recommendations are implemented as a basic structure and cannot adapt to the behavior of real users. Also, input validation is limited and might be improved to allow better user experience and forms. Furthermore, optional modules like Pillow and tkintermapview might not be installed in every environment and cause errors. Besides, the number of restaurants in the dataset is too low. Some improvements for future development might include adding real user accounts, implementing favorite restaurants list, adding stronger authentication, learning from the behavior of users to make recommendations, using a larger dataset of restaurants, and advanced search capabilities.

Appendix A. Module Summary

Module	Main Responsibility	Key Examples
main.py	Program entry point	Creates UniEatsApp and starts mainloop
gui.py	User interface and event handling	Filtering, review submission, admin actions, map display
database.py	SQLite persistence layer	Create tables, seed data, CRUD on restaurants/reviews
services.py	Business logic and map services	Filtering, sorting, graph-based recommendation, map markers
models.py	Domain models and OOP structure	Person/User/Admin, Restaurant, Review, FeaturedRestaurant
utils.py	Reusable helper functions/classes	DistanceCalculator, ShellSort, image copy utility

Appendix B. OOP Concept-to-Code Evidence

OOP concept	Where shown	Illustration	Comment
Class/Object	models.py, gui.py, database.py, services.py	Application built from interacting classes and objects	Clearly demonstrated
Abstraction	Person(ABC)	Abstract method get_role() defines required interface	Matches course abstraction topic
Encapsulation	Person._username + property	Internal attribute accessed through property	Present but could be extended
Inheritance	User/Admin from Person; FeaturedRestaurant from Restaurant	Child classes reuse parent structure	Clearly demonstrated
Polymorphism	get_role(), display_text()	Same interface with different subclass behaviour	Implemented, but could be used more in workflow
Composition	UniEatsApp and RestaurantService	Classes own and coordinate other objects	Improves modularity

Appendix C. Declared External Resources

Resource	Type	Purpose
tkinter	Standard library package	Desktop graphical user interface
sqlite3	Standard library package	Local relational database
abc	Standard library package	Abstract base class support
os, shutil, random, math	Standard library modules	File handling, utilities, randomness, distance formula
Pillow	External package	Image preview support
tkintermapview	External package	Embedded OpenStreetMap display

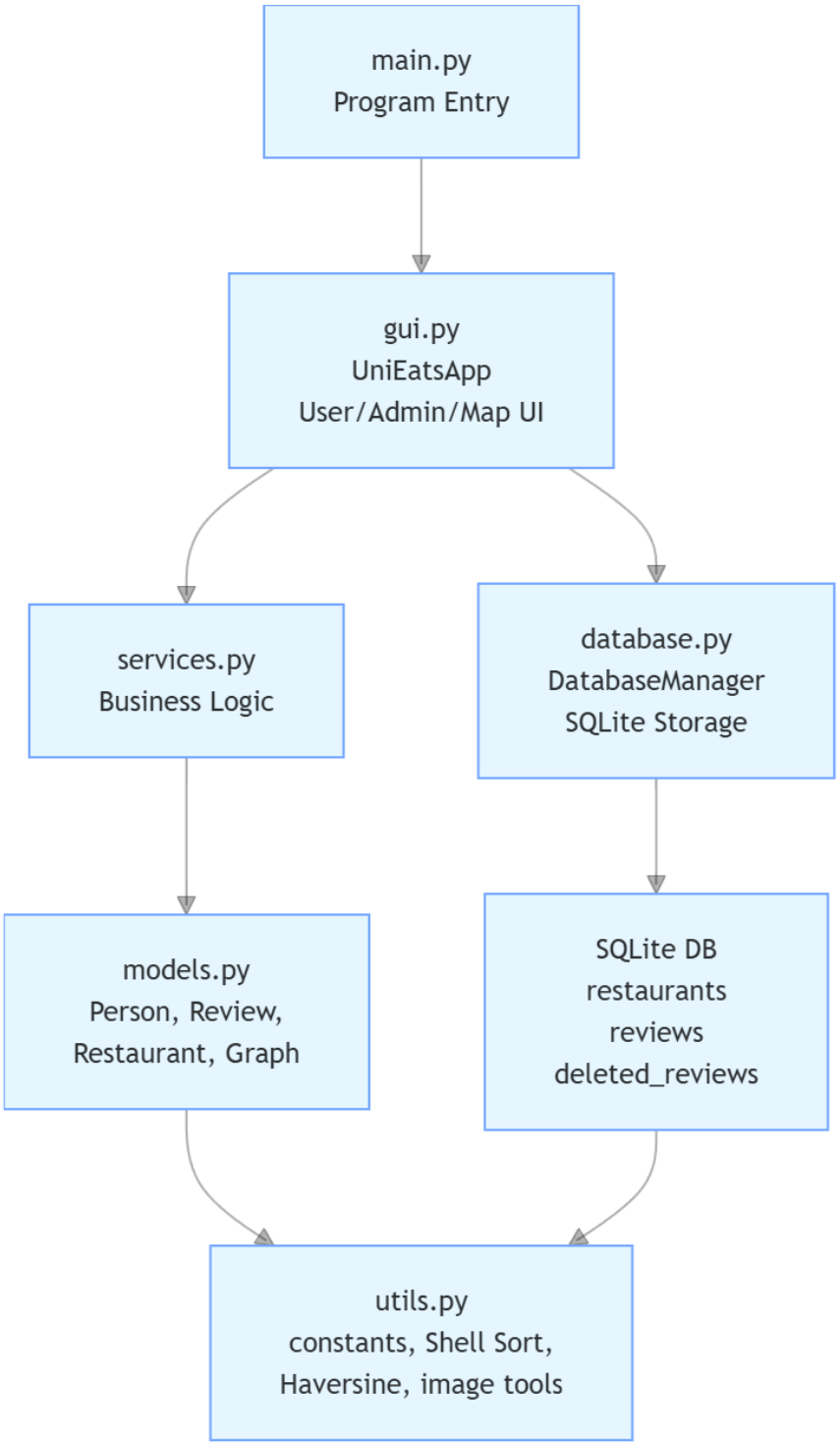
Appendix D. References

COMP 2090SEF/8090SEF Lecture Note 1: Advanced Python Programming: List Comprehension and Lambda Function.
COMP 2090SEF/8090SEF Lecture Note 2: Object-Oriented Programming (OOP) and Python Class.
COMP 2090SEF/8090SEF Lecture Note 3: Recursion, Searching and Sorting Algorithms.
COMP 2090SEF/8090SEF Lecture Note 4: Introduction to Algorithmic Complexity.
COMP 2090SEF/8090SEF Lecture Note 5: Stack, Queue, and Linked List.
COMP 2090SEF/8090SEF Lecture Note 6: Hash Tables.
COMP 2090SEF/8090SEF Lecture Note 7: Binary Trees.
Submitted source files: main.py, gui.py, database.py, services.py, models.py, utils.py.

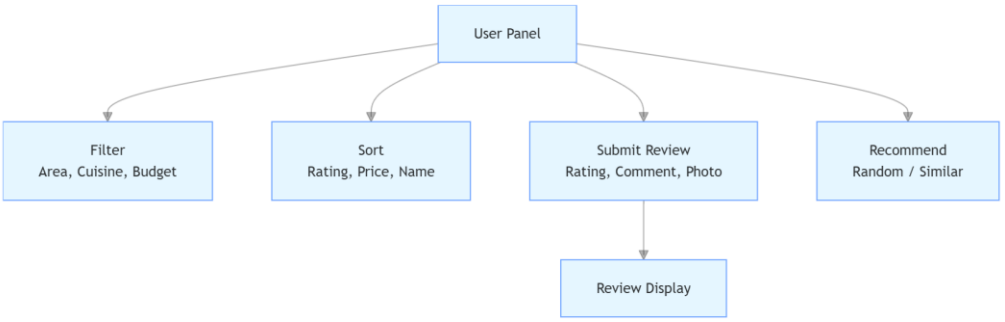
Appendix E. Github link

https://github.com/owounnamed/COMP-2090SEF-GRP_105-project

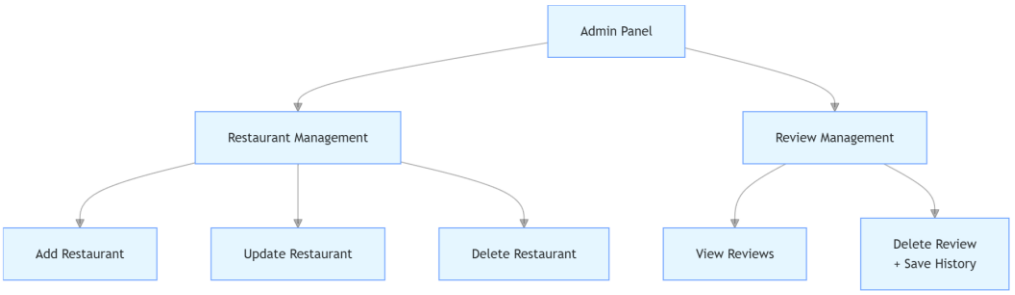
Appendix F. Overall System Architecture



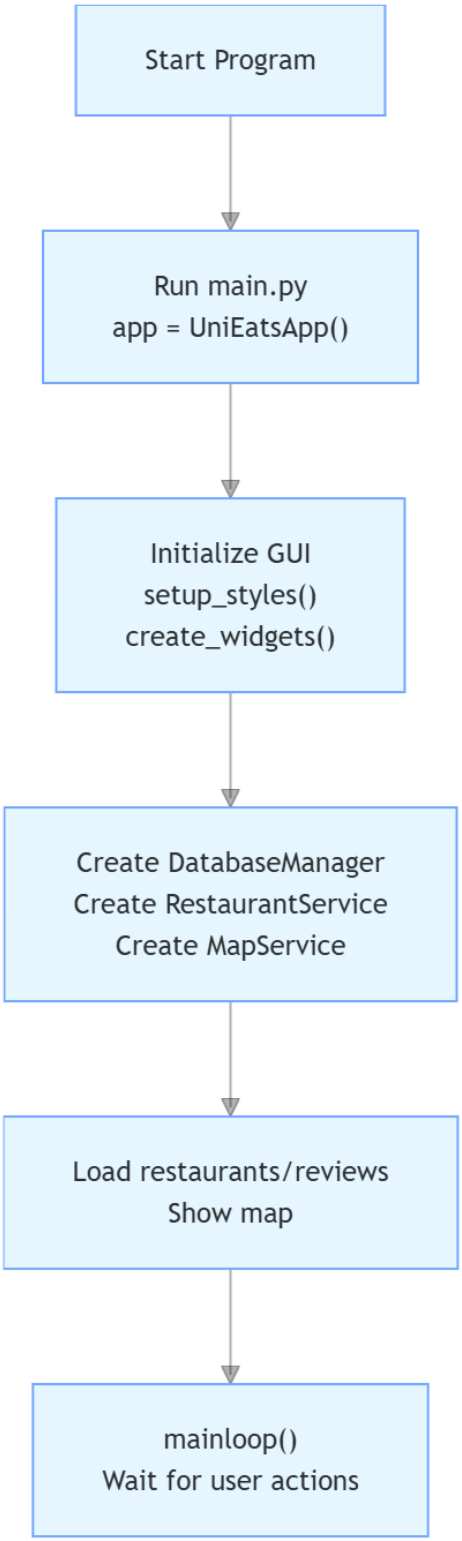
Appendix G. User-Side Functional Structure



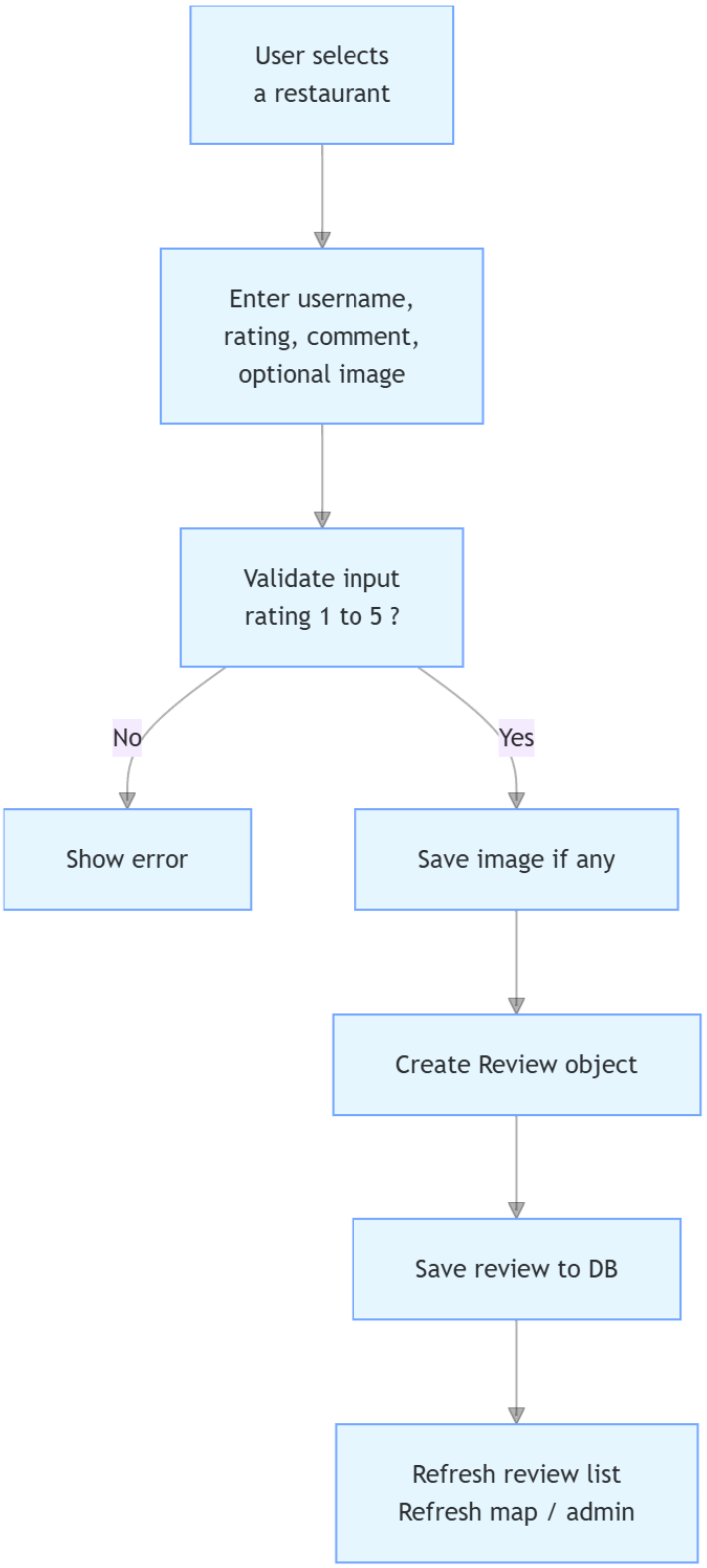
Appendix H. Admin Functional Structure



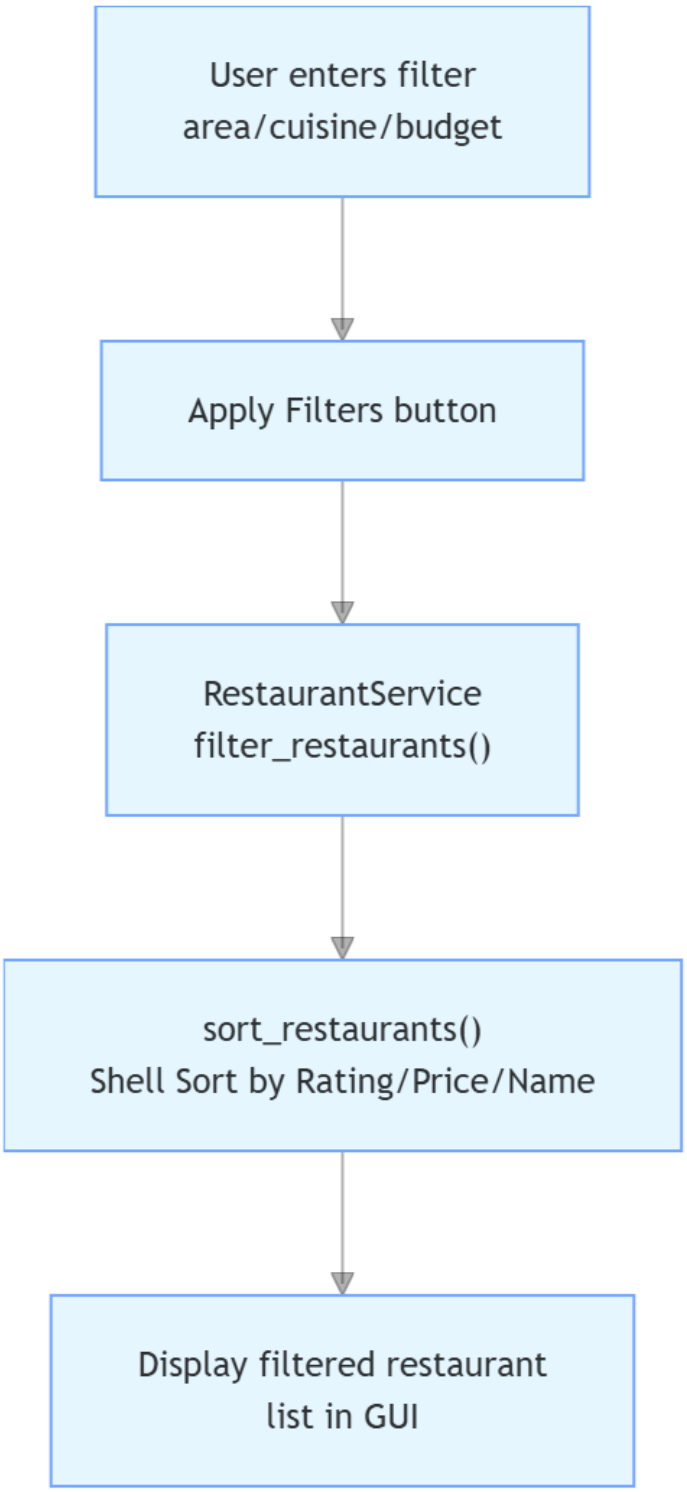
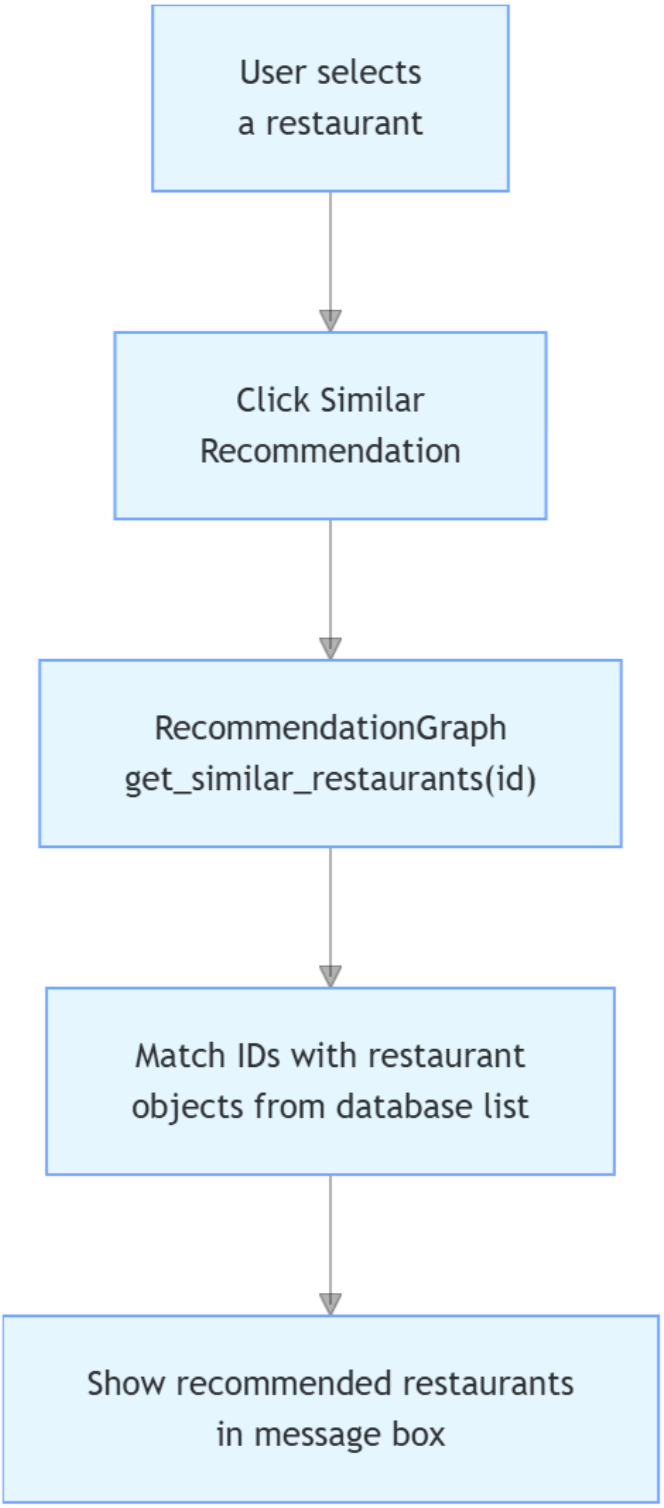
Appendix I. Flowchart: Program Startup



Appendix J. Flowchart: Review Submission Process



Appendix K. Flowchart: Restaurant Filtering and Sorting



Appendix L. Flowchart: Similar Recommendation Process